



第04章 数学函数、字符和字符串

张仁斌@合肥工业大学软件学院

本章主要内容



4.1 数学函数



4.2 字符数据类型和操作符

4.3 字符函数

4.4 字符串类型

4.5 格式化控制台输出

4.6 简单的文件输入输出

4.7 作业与实践

cmath中的数学函数



◆ C/C++在头文件**cmath**中提供了许多有用的、常见的数学函数

函数类型	函数名	描述
三角函数	sin(radians)	返回以 弧度 表示的角度的正弦值
	cos(radians)	返回以弧度表示的角度的余弦值
	tan(radians)	返回以弧度表示的角度的正切值
	asin(a)	返回正弦函数的弧度角度值
	acos(a)	返回余弦函数的弧度角度值
	atan(a)	返回正切函数的弧度角度值
指数函数	exp(x)	返回 e^x 的值
	log(x)	返回自然对数的值 ($\log_e(x)$)
	log10(x)	返回以10为底的对数的值 ($\log_{10}(x)$)
	pow(a, b)	返回 a^b 的值
	sqrt(x)	返回x的平方根，当 $x \geq 0$ 时
近似函数	ceil(x)	x被向上取证为一个最接近它的整数（以double类型），如ceil(5.1)为6.0，ceil(-5.8)为-5.0
	floor(x)	x被向下取证为一个最接近它的整数（以double类型），如floor(5.8)为5.0，floor(-5.1)为-6.0

不是四舍五入，小数部分不为0时则绝对值加1或减1

min、max、abs函数



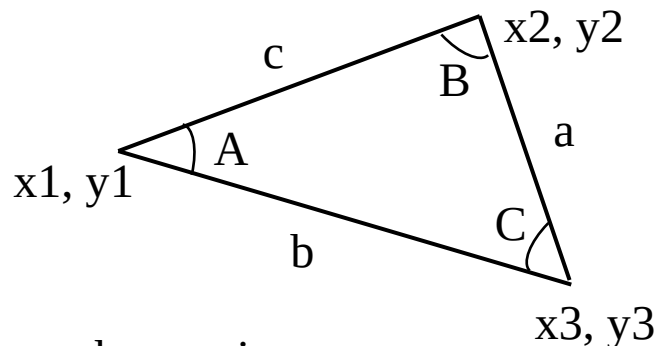
- ◆ **min**返回两个数整数或浮点数中的最小值，**max**则返回最大值，例如：
 - `min(2.5, 3.0)`返回值为2.5（函数的参数为2.5和3.0时，函数的值为2.5）
 - `max(2.5, 3.0)`返回值为3.0
- ◆ **abs**函数参数的绝对值，参数可以是整数（**int**, **long**）或浮点数（**float**, **double**），例如
 - `abs(-2)`返回值为2
 - `abs(-2.1)`返回值为2.1
- ◆ 在GNU C/C++中，**min**、**max**、**abs**函数定义在**cstdlib**头文件中，而在Visual C++中，函数定义在**algorithm**头文件中
 - **cstdlib**: C++ standard library

示例：计算三角形的角



◆ 编写程序提示用户输入三角形三个顶点的坐标，计算并显示该三角形三个角的角度（以度为单位）

```
#include <iostream>
#include <cmath>
using namespace std;
```



```
int main()
{
    // Prompt the user to enter three points
    cout << "Enter the coordinates of three points separated "
          << "by spaces like x1 y1 x2 y2 x3 y3: ";
    double x1, y1, x2, y2, x3, y3;
    cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3;

    // Compute three sides
    double a = sqrt((x2 - x3) * (x2 - x3) + (y2 - y3) * (y2 - y3));
    double b = sqrt((x1 - x3) * (x1 - x3) + (y1 - y3) * (y1 - y3));
    double c = sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2));
```

```
// Obtain three angles in radians
double A = acos((a * a - b * b - c * c) / (-2 * b * c));
double B = acos((b * b - a * a - c * c) / (-2 * a * c));
double C = acos((c * c - b * b - a * a) / (-2 * a * b));

// Display the angles in degrees
const double PI = 3.14159;
cout << "The three angles are " << A * 180 / PI << " "
      << B * 180 / PI << " " << C * 180 / PI << endl;

return 0;
}
```

思考使用常量的优点

D:\Edu-CPP\chap04\ex04-01.exe

```
Enter the coordinates of three points separated by
spaces like x1 y1 x2 y2 x3 y3: 1 1 6.5 1 6.5 2.5
The three angles are 15.2551 90.0001 74.7449
```

本章主要内容



4.1 数学函数

4.2 字符数据类型和操作符



4.3 字符函数

4.4 字符串类型

4.5 格式化控制台输出

4.6 简单的文件输入输出

4.7 作业与实践



字符数据类型

◆ 字符数据类型（**char**）代表一个**单独**的字符

◆ 一个字符的文字被单引号括起来

```
char letter = 'A';    // 定义字符变量并赋值字母字符A
```

```
char numChar = '6'; // 定义字符变量并赋值数字字符6
```

```
cout << ++letter;    // 输出字符B
```

—— 字符采用**ASCII码**，每个字符对应一个正整数

■ 注意： "A"是一个字符串， 'A'是一个字符

ASCII码



- ◆ 计算机使用二进制，一个字符在计算机中存储为多个0和1。把一个字符映射为二进制码称作编码。大多数计算机使用ASCII（**A**merican **S**tandard **C**ode for **I**nformation **I**nterchange，美国信息交换标准代码）
- ◆ ASCII码是一种8位（1字节）的编码方案表示所有的大小写字母、数字、标点符号和控制字符。常见字符的ASCII码值如表所示（完整的ASCII码表详见群共享）

字符	ASCII码值
'0' ~ '9'	48~57
'A' ~ 'Z'	65~90
'a' ~ 'z'	97~122

- ◆ 从键盘读入一个字符

```
char ch;  
cout << "Enter a character: ";  
cin >> ch;  
cout << "The character read is " << ch << endl;
```




特殊字符的转义字符

- ◆ 引例：如果要显示带引号的输出，可以用类似下面的语句吗？

```
cout << "He said \"Programming is fun!\"" << endl;
```

存在编译错误，编译器认为这是字符串的结束，不知如何处理余下的字符

- ◆ 为了能使用特殊字符，C/C++使用转义字符代表特殊字符，如表所示

- 转义字符也称作转义序列，由一个反斜杠（转义符）和一个字符或数字组合而成，即斜杠后面的字符被转义了，不是字符本身

- 转义序列被解释为一个整体，视作一个字符

```
cout << "He said \"Programming is fun!\"" << endl;
```

D:\Edu-CPP\chap04\ex04-02.exe

He said "Programming is fun!"

字符	名称	ASCII码值	字符	名称	ASCII码值
\b	回退符	8	\r	回车符	13
\t	制表符	9	\\	反斜杠	92
\n	换行符	10	\"	双引号	34
\f	换页符	11			

转义字符应用示例



```
#include <iostream>
using namespace std;
```

```
int main()
{
```

```
    cout << "\\t is a tab character" << endl << endl;
```

```
    cout << "Welcome to C++\\n";
```

```
    cout << "Welcome to C++" << endl;
```

```
    cout << "abc\\ndef" << endl << endl;
```

```
    cout << "a\\bcdef" << endl;
```

```
    cout << "[\\\"Y\\b=\\\"]\\101 is " << "[Y\\b=]\\101" << endl;
```

```
    return 0;
```

```
}
```

D:\Edu-CPP\chap04\ex04-02B.exe

\t is a tab character

Welcome to C++

Welcome to C++

abc

def

cdef

["Y\b="]\101 is [=]A

\ddd, 1到3位八进制数所代表的字符
\xhh, 十六进制数所代表的字符

表达式 '\101' + 0xa1 - (int)3.6 * 2 的值为?

数值类型和字符类型之间的转换



- ◆ 一个整数转换为字符时，只有其低8位用于转换为字符，其余被忽略

```
char c = 0xFF41; // 低8位即0x41赋给字符变量c
```

```
cout << c;      // 显示字符A
```

- ◆ 一个浮点数转换为字符时，浮点数先转换为整数，再转为字符类型

```
char c = 65.25;  // 将65赋给c
```

```
cout << c;      // 显示字符A
```

- ◆ 一个字符转换为数值类型时，字符的ASCII码值被转换到指定的数值变量中

```
int i = 'A';     // i为65
```

```
cout << i;      // 显示65
```



◆ **char**类型被视作1字节长度的整数，数值运算符都可以用于字符操作，当数值运算符的操作数是字符时，字符自动转换为数字，例如：

```
#include <iostream>
using namespace std;

int main()
{
    // '2'的ASCII码为50, '3'的ASCII码为 51
    int i = '2' + '3';
    cout << "i is " << i << endl;

    int j = 2 + 'a'; // 'a'的ASCII码为97
    cout << "j is = " << j << endl;
    cout << j << " is the ASCII code for character " << static_cast<char>(j) << endl;

    return 0;
}
```

D:\Edu-CPP\chap04\ex04-03.exe

```
i is 101
j is = 99
99 is the ASCII code for character c
```



◆ ASCII码中大写字母的值是连续整数，小写字母的值也是连续整数，且'a'的值比'A'的值大32，可以利用这些特点实现大小写字母的转换

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Enter a lowercase letter: ";
    char lowercaseLetter;
    cin >> lowercaseLetter;

    char uppercaseLetter = static_cast<char>('A' + (lowercaseLetter - 'a'));

    cout << "The corresponding uppercase letter is " << uppercaseLetter << endl;

    return 0;
}
```

如何判断输入的是否是小写字母？

D:\Edu-CPP\chap04\ex04-04.exe

```
Enter a lowercase letter: j
The corresponding uppercase letter is J
```

比较和测试字符



```
#include <iostream>
using namespace std;
int main() {
    cout << "Enter a letter: ";
    char ch;
    cin >> ch;

    if(ch >= 'A' && ch <= 'Z')
        cout << ch << " is an uppercase letter" << endl;
    else if(ch >= 'a' && ch <= 'z')
        cout << ch << " is a lowercase letter" << endl;
    else if(ch >= '0' && ch <= '9')
        cout << ch << " is a numeric character" << endl;

    cout << "'a' < 'b' is " << ('a' < 'b') << endl;
    cout << "'a' < 'A' is " << ('a' < 'A') << endl;
    cout << "'1' < '8' is " << ('1' < '8') << endl;
    return 0;
}
```

D:\Edu-CPP\chap04\ex04-05.exe

```
Enter a letter: j
j is a lowercase letter
'a' < 'b' is 1
'a' < 'A' is 0
'1' < '8' is 1
```

示例：生成随机字符



- ◆ 每个字符都是一个介于0~127之间的唯一的ASCII码，生成随机字符即随机生成一个介于0~127的随机整数。可以参照先前生成随机数的算法
 - $a + \text{rand}() \% b$ 得到一个介于 $a \sim a+b$ 的随机整数，包括 $a+b$
 - $\text{static_cast<char>}(ch1 + \text{rand}() \% (ch2 - ch1 + 1))$ 得到介于 $ch1$ 和 $ch2$ 之间的随机字符 ($ch1 \leq ch2$)



◆ 给定两个字符，随机生成介于二者之间的字符

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;
int main() {
    cout << "Enter a starting character: ";
    char startChar;
    cin >> startChar;
    cout << "Enter an ending character: ";
    char endChar;
    cin >> endChar;

    // Get a random character
    srand(time(0));
    char randomChar = static_cast<char>(startChar + rand() % (endChar - startChar + 1));;
    cout << "The random character between " << startChar << " and " << endChar << " is " << randomChar << endl;
    return 0;
}
```

D:\Edu-CPP\chap04\ex04-06.exe

```
Enter a starting character: a
Enter an ending character: m
The random character between a and m is k
```

D:\Edu-CPP\chap04\ex04-06.exe

```
Enter a starting character: z
Enter an ending character: a
The random character between z and a is
```

如何确保endChar >= startChar?

本章主要内容



4.1 数学函数

4.2 字符数据类型和操作符

4.3 字符函数



4.4 字符串类型

4.5 格式化控制台输出

4.6 简单的文件输入输出

4.7 作业与实践



◆ C/C++包含了许多测试字符和转换字符的函数，在头文件cctype中定义

■cctype: check character type

函数	描述	函数	描述
isdigit(ch)	仅当字符ch是数字时返回true	isupper(ch)	仅当字符ch是大写字母时返回true
isalpha(ch)	仅当字符ch是字母时返回true	isspace(ch)	仅当字符ch是空白字符时返回true
isalnum(ch)	仅当字符ch是字母或数字时返回true	tolower(ch)	返回字符ch的小写形式
islower(ch)	仅当字符ch是小写字母时返回true	toupper(ch)	返回字符ch的大写形式

```
isdigit('9');    // 返回函数值为true
isdigit('a');    // 返回函数值为false;
isalnum('8');    // 返回函数值为true;
islower('A');    // 返回函数值为false
```

示例：判断输入字符的类型



```
#include <iostream>
#include <cctype>
using namespace std;
int main() {
    cout << "Enter a character: ";
    char ch;
    cin >> ch;

    if (islower(ch)) {
        cout << "It is a lowercase letter " << endl;
        cout << "Its equivalent uppercase letter is " << static_cast<char>(toupper(ch)) << endl;
    }
    else if (isupper(ch)) {
        cout << "It is an uppercase letter " << endl;
        cout << "Its equivalent lowercase letter is " << static_cast<char>(tolower(ch)) << endl;
    }
    else if (isdigit(ch))
        cout << "It is a digit character " << endl;
    return 0;
}
```

```
D:\Edu-CPP\chap04\ex04-07.exe

Enter a character: h
It is a lowercase letter
Its equivalent uppercase letter is H
```

示例：个位十六进制数转换为十进制



十六进制数系统有16个数字：0~9，A~F。A~F对应十进制的10~15

```
#include <iostream>
#include <cctype>
using namespace std;
int main() {
    cout << "Enter a hex digit: ";
    char hexDigit;
    cin >> hexDigit;

    hexDigit = toupper(hexDigit);
    if (hexDigit <= 'F' && hexDigit >= 'A')
    {
        int value = 10 + hexDigit - 'A';
        cout << "The decimal value for hex digit " << hexDigit << " is " << value << endl;
    }
    else if (isdigit(hexDigit))
        cout << "The decimal value for hex digit " << hexDigit << " is " << hexDigit << endl;
    else
        cout << hexDigit << " is an invalid input" << endl;
    return 0;
}
```

为什么要转换为大写？

D:\Edu-CPP\chap04\ex04-08.exe

```
Enter a hex digit: f
The decimal value for hex digit F is 15
```

D:\Edu-CPP\chap04\ex04-08.exe

```
Enter a hex digit: 8
The decimal value for hex digit 8 is 8
```

D:\Edu-CPP\chap04\ex04-08.exe

```
Enter a hex digit: h
H is an invalid input
```

本章主要内容



4.1 数学函数

4.2 字符数据类型和操作符

4.3 字符函数

4.4 字符串类型



4.5 格式化控制台输出

4.6 简单的文件输入输出

4.7 作业与实践

字符与字符串



◆ **char**数据类型仅代表一个字符，可以使用**string**数据类型代表一个字符串

■ 例如： `string message = "Programming is fun!";` // message的值或内容是Prog.....

■ 一个字符串是一序列的字符

◆ **string**是C++新增的数据类型，是一种**对象类型**（**object type**），当定义一个对象类型的变量时，实际是创建一个对象

■ 对象是利用“**类**（**class**）”定义的，string是一个预定义在**<string>头文件**中的类，一个对象是一个类的**实例**

➤ `string message = "Programming is fun!";` 创建了string类的对象（实例）message

■ 对象和类，将在第9章介绍



string对象的简单函数

◆ **string**类的函数只能被特定的**string**实例调用，调用语法是：

■ “实例对象.实例函数(函数参数)”，例如：message.length()；

➤ 一个函数可能有多个参数，也有可能没有任何参数

◆ **string**对象的简单函数如表所示

函数	描述
length()	返回字符串中的字符个数
size()	同length()
at(index)	返回字符串中指定位置的字符

string类型简单示例



```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string message = "How are you?";
    cout << message.length() << endl; // 输出12
    cout << message.at(0) << endl;    // 输出字符H
    string s; // 定义了一个string类型的变量s，默认初值为空字符串，即等同于： string s = "";
    s = "ABCD";
    s[1] = 'P'; // 修改单个字符
    cout << s << endl;
    cout << s.length() << endl; // 输出4
    cout << s.at(1) << endl;    // 输出字符P
    return 0;
}
```

C/C++中，索引序号是从0开始的，
即：字符串第1个字符的索引是0

```
D:\Edu-CPP\chap04\ex04-09.exe
12
H
APCD
4
P
```


字符串索引和下标操作符



- ◆ `s.at(index)` 函数访问字符串 `s` 中索引 `index` 指定的字符，索引值介于 `0~s.length()-1`，即，`string` 对象中的字符可以通过其索引来访问

索引	0	1	2	3	4	5	6	7	8	9	10	11	12	13
message	W	e	l	c	o	m	e		t	o		C	+	+

↑ `message.at(0)` `message.length()` 为 14 ↑ `message.at(13)`

- ◆ C++ 提供 **下标操作符**（即使用 `stringName[index]` 函数）**检索或修改** 字符串中指定位置的字符

```
string s = "ABCD";  
s[1] = 'P'; // s = "APCD"  
cout << s[1] << endl;
```

访问字符串 `s` 中越界字符是一个常见编程错误，例如：

`s.at(s.length())`

或

`s[s.length()]`

字符串的连接



◆ C++中可以用+操作符连接两个字符串，或将一个字符与一个字符串相连接

■但不能直接连接两个字符串常量

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main()
```

```
{
```

```
    string s = "Beijing";
```

```
    s += ',';
```

```
    string cities = s + "Shanghai";
```

```
    cout << cities << endl;
```

```
    //string cities2 = "Beijing," + "Shanghai";
```

```
    return 0;
```

```
}
```

```
D:\Edu-CPP\chap04\ex04-10.exe
```

```
Beijing,Shanghai
```

“+”是左结合的，将字符或字符串转换为“+”左边的string类型再连接

进行“+”运算的两个操作数都是字符串常量（不具有string类型特征），违反“+”运算规则，故是一个编译错误

字符串的比较



- ◆ 可以使用关系运算符<、<=、>、>=、==、!=比较两个字符串，具体的比较规则是两个字符串从左到右每个字符对应比较

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s1 = "ABC123";
    string s2 = "ABE123";
    cout << (s1 == s2) << endl; // 0: false
    cout << (s1 != s2) << endl; // 1: true
    cout << (s1 > s2) << endl; // 0: false
    cout << (s1 >= s2) << endl; // 0: false
    cout << (s1 < s2) << endl; // 1: true
    cout << (s1 <= s2) << endl; // 1: true

    return 0;
}
```

发现已不满足“==”了，还需要继续比较后面的数字字符吗？
(优化关系运算的实现)

```
D:\Edu-CPP\chap04\ex04-11.exe
0
1
0
0
1
1
1
```

读取字符串



- ◆ 可以使用cin从键盘读取字符串，但字符串不能包含空格（空格表示英文单词结束而结束字符串输入）。若需读取包含空格的字符串，则使用<string>头文件中提供的getline函数：**getline(cin, s, delimiterCharacter)**

➤ 其中**分隔符**delimiterCharacter是默认的'\n'时，可以省略

```
#include <iostream>
#include <string>
#include <limits>
using namespace std;
int main() {
    string city, city2;
    cout << "Enter a city: ";
    cin >> city;
    cout << "You entered " << city << endl;
    cout << "Enter a city: ";
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    getline(cin, city2);
    cout << "You entered " << city2 << endl;
    return 0;
}
```

```
D:\Edu-CPP\chap04\ex04-12.exe
Enter a city: He Fei
You entered He
Enter a city: He Fei
You entered He Fei
```

若无此语句，不清除cin缓冲区，则：

```
D:\Edu-CPP\chap04\ex04-12.exe
Enter a city: He Fei
You entered He
Enter a city: You entered Fei
```

示例：字符串录入与排序



◆ 编写程序提示用户输入两个城市名（拼音或英语），并升序显示

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string city1, city2;
    cout << "Enter the first city: ";
    getline(cin, city1);
    cout << "Enter the second city: ";
    getline(cin, city2);

    cout << "The cities in alphabetical order are ";
    if (city1 < city2)
        cout << city1 << " & " << city2 << endl;
    else
        cout << city2 << " & " << city1 << endl;

    return 0;
}
```

D:\Edu-CPP\chap04\ex04-13.exe

```
Enter the first city: Shang Hai
Enter the second city: He Fei
The cities in alphabetical order are He Fei & Shang Hai
```

本章主要内容



4.1 数学函数

4.2 字符数据类型和操作符

4.3 字符函数

4.4 字符串类型

4.5 格式化控制台输出



4.6 简单的文件输入输出

4.7 作业与实践

引例



- ◆ 计算结算利息等场景，小数点后只需显示两位，采用乘以100取整后除以100.0的方法，难以满足要求（尾部为0时不显示0，精度不足），可以采用流操作在控制台显示格式化输出流

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
int main() {
    double amount = 12618.98;
    double interesRate = 0.0013;
    double interest = amount * interesRate;

    cout << "Interest is " << interest << endl; // 存在不需要的小数位
    cout << "Interest is " << static_cast<int>(interest * 100) / 100.0 << endl; // 精度不足
    cout << "Interest is " << fixed << setprecision(2) << interest << endl;

    return 0;
}
```

D:\Edu-CPP\chap04\ex04-14.exe

```
Interest is 16.4047
Interest is 16.4
Interest is 16.40
```



常用的流操作

◆使用cout对象在控制台显示输出，C++提供附加函数格式化要显示的值，这些函数称作流操作，包含在<iomanip>头文件中

■iomanip: **io manipulator**

操作	描述
setprecision(n)	设定输出浮点数的精度
fixed	设定浮点数按指定小数位数显示（默认6位小数，可以用setprecision变更）
showpoint	即使没有小数部分，也显示以0补足的小数点后位数
setw(width)	输出信息占指定宽度width（若信息实际长度大于width，则以实际长度为准）
left	左对齐输出
right	右对齐输出

◆流操作仅对设置操作之后的输出有影响，此前已输出的信息不受此操作影响



setprecision(n)

◆ **只使用** `setprecision(n)` 时，设置 **浮点数** 总的显示位数 **n**（显示精度 **n**），即小数点前后位数的总和，浮点数采用 **四舍五入** 近似值显示

➤ `fixed` **配合使用** `setprecision(n)` 时，`n` 为小数点后位数；下面示例未使用 `fixed`

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    double number = 12.34567;
    double pi = 3.1415926;

    cout << setprecision(3) << number << endl;
    cout << setprecision(4) << number << endl;
    cout << setprecision(5) << number << endl;
    cout << setprecision(6) << number << endl;
    cout << pi << endl;
    cout << setprecision(3) << 12345.6 << endl; // 整数部分长度大于精度，用科学计数法
    return 0;
}
```

// 沿用此前的设置

// **整数部分长度大于精度，用科学计数法**

如果改为整数123456，显示结果如何？

```
D:\Edu-CPP\chap04\ex04-15.exe
12.3
12.35
12.346
12.3457
3.14159
1.23e+004
```

fixed



◆ 计算机可能会自动用科学计数法显示一个很长的浮点数，可以使用 **fixed** 操作强制显示为非科学计数法的形式，默认显示6位小数

■ **fixed** 配合使用 **setprecision(n)** 指定小数位数 **n**

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    double number = 123456789.4567;

    cout << number << endl;           // Windows系统默认用科学计数法
    cout << setprecision(3) << number << endl; // 执行fixed之前
    cout << fixed << number << endl;         // 按此前指定小数位数
    cout << setprecision(2) << number << endl; // 按新指定小数位数

    return 0;
}
```

D:\Edu-CPP\chap04\ex04-16.exe

```
1.23457e+008
1.23e+008
123456789.457
123456789.46
```

showpoint



◆默认情况下，没有小数部分的浮点数是不显示小数点的，可以使用**fixed**操作强制浮点数显示小数点和指定的小数位数，也可以使用**showpoint**和**setprecision**一起解决问题，并用0补足位数

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    cout << setprecision(6);
    cout << 1.23 << endl; // 实际有效位数不足默认的6，原样输出
    cout << 123.0 << endl; // 小数部分不显示，即显示为整数
    cout << showpoint << 1.23 << endl; // 强制按小数和整数部分总6位
    cout << showpoint << 123.0 << endl; // 同上
    cout << 123.45 << endl;           // 延用已有设置

    return 0;
}
```

```
D:\Edu-CPP\chap04\ex04-17.exe

1.23
123
1.23000
123.000
123.450
```



setw(width)

◆ 默认情况下，`cout`只使用所需输出位数占用的显示宽度，可以使用`setw(width)`函数指定输出的**最小**宽度`width`

■ 若实际输出内容超过指定的宽度，则按实际内容宽度

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
int main()
{
```

```
    cout << "C++" << 123 << endl; // 按内容所需宽度
    cout << setw(8) << "C++" << setw(6) << 123 << endl; // 按指定宽度
    cout << setw(8) << "HelloZRB" << setw(6) << 12345 << endl; // 同上
    cout << setw(8) << "C++" << 123 << endl; // setw仅影响随后的一次输出
    cout << setw(5) << "HelloZRB" << endl; // 忽略宽度设置，按实际大小
```

```
    return 0;
```

```
}
```

C + + 1 2 3											
					C	+	+			1	2 3
H	e	l	l	o	Z	R	B		1	2	3 4 5
					C	+	+		1	2	3
H	e	l	l	o	Z	R	B				

D:\Edu-CPP\chap04\ex04-18.exe

C++123

C++ 123
HelloZRB 12345

C++123
HelloZRB

setw操作默认使输出内容
右对齐，如何改为左对齐？



left和right

◆ **setw**操作默认使用右对齐，可以使用**left**和**right**操作设置输出为左对齐或右对齐

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    cout << setw(8) << "C++" << setw(6) << 123 << endl;           // 默认右对齐
    cout << setw(8) << "HelloZRB" << setw(6) << 12345 << endl; // 同上
    cout << left;
    cout << setw(8) << "C++";           // 左对齐输出，且不换行
    cout << right << setw(6) << 123 << endl; // 右对齐输出

    return 0;
}
```

					C	+	+				1	2	3
H	e	l	l	o	Z	R	B		1	2	3	4	5
C	+	+									1	2	3

```
D:\Edu-CPP\chap04\ex04-19.exe
      C++      123
HelloZRB 12345
C++          123
```



示例：显示三角函数计算表

◆编写程序显示如下三角函数计算表

Degrees	Radians	Sine	Cosine	Tangent
30	0.5236	0.5000	0.8660	0.5773
60	1.0472	0.8660	0.5000	1.7320

- 使用`setw(width)`控制每列每个数据的**统一宽度**（实现**纵向对齐**），使用`setprecision(n)`控制浮点数小数位数
- 逐行逐列输出
- 角度值（度或弧度）都是`double`类型的
- 度转换为弧度时，`PI`定义值为3.14159的常量

```
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;
int main() {
```

```
// Display the header of the table
```

```
cout << left << setw(10) << "Degrees" << setw(10) << "Radians"
    << setw(10) << "Sine" << setw(10) << "Cosine" << setw(10) << "Tangent" << endl;
```

```
// Display values for 30 degrees
```

```
const double PI = 3.14159;
double degrees = 30;
double radians = degrees * (PI / 180);
cout << setw(10) << degrees << setw(10) << fixed << setprecision(4) << radians
    << setw(10) << sin(radians) << setw(10) << cos(radians) << setw(10) << tan(radians) << endl;
```

```
// Display values for 60 degrees
```

```
degrees = 60;
radians = degrees * (PI / 180);
cout << setw(10) << setprecision(0) << degrees << setw(10) << setprecision(4) << radians
    << setw(10) << sin(radians) << setw(10) << cos(radians) << setw(10) << tan(radians) << endl;
return 0;
}
```

D:\Edu-CPP\chap04\ex04-20.exe

Degrees	Radians	Sine	Cosine	Tangent
30	0.5236	0.5000	0.8660	0.5773
60	1.0472	0.8660	0.5000	1.7320

输出30度值之前，为什么可以不设置输出精度？
输出60度值之前，为什么要设置小数位数为0？

本章主要内容



4.1 数学函数

4.2 字符数据类型和操作符

4.3 字符函数

4.4 字符串类型

4.5 格式化控制台输出

4.6 简单的文件输入输出



4.7 作业与实践

写入文件



- ◆ 可以使用cin对象读取键盘输入，用cout对象输出到控制台，也可以从文件中读取数据或向文件中写入（存储）数据
- ◆ 向文件中写入数据，基本流程是：
 - 使用在**头文件<fstream>**中定义的**ofstream**类型（对象类型）定义一个变量，即创建一个ofstream类的对象，
ofstream output;
 - 利用该**对象的open函数**打开目标文件
output.open("numbers.txt"); // 若文件不存在，则新建，否则，覆盖文件内容
 - 利用流插入操作符（<<）向文件中写入数据，类似把数据发送到cout对象
output << 95 << " " << 56 << " " << 34;
 - 写文件完毕，不需再写文件时，调用**对象的close函数**关闭文件
output.close();
- ◆ 教材第13章给出了文件读写的详细操作

写文件示例



```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main() {
    ofstream output;
```

// 若文件不存在，则新建，否则，覆盖文件内容

```
output.open("numbers.txt");
```

// Write numbers

```
output << 95 << " " << 56 << " " << 34;
```

// close file

```
output.close();
```

```
cout << "Done" << endl;
```

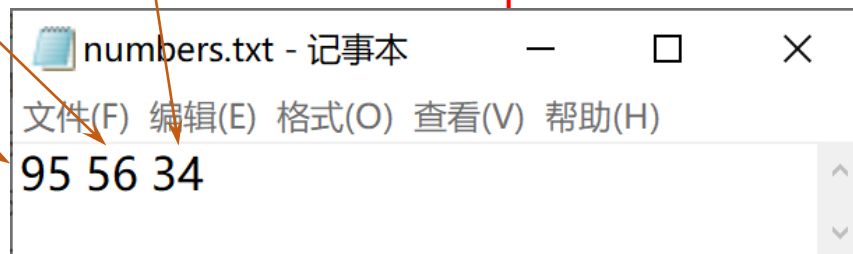
```
return 0;
```

```
}
```

D:\Edu-CPP\chap04\ex04-21.exe

Done

文件与编译生成的exe文件在同一文件夹，可以指定路径，例如：
d:\\numbers.txt （注意：Windows系统中c盘根目录有权限限制）



读文件



◆从文件中读入数据的基本流程是：

- 使用在**头文件<fstream>**中定义的**ifstream**类型（对象类型）定义一个变量，即创建一个ifstream类的对象，

```
ifstream input;
```

- 利用该**对象的open函数**打开目标文件

```
input.open("numbers.txt"); // 若文件不存在，则报错
```

- 利用输出流操作符（>>）从文件中读取数据，类似用cin对象读取数据

```
input >> score1;
```

- 读文件完毕，不需再读文件时，调用**对象的close函数**关闭文件

```
input.close();
```

读文件示例



```
#include <iostream>
#include <fstream>
using namespace std;

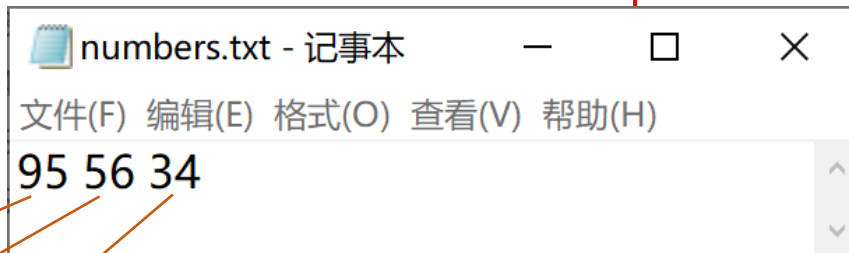
int main() {
    ifstream input;

    // Open a file
    input.open("numbers.txt");

    // Read data
    int score1, score2, score3;
    input >> score1 >> score2 >> score3;
    cout << "Total score is " << score1 + score2 + score3 << endl;

    // Close file
    input.close();

    return 0;
}
```



D:\Edu-CPP\chap04\ex04-22.exe

Total score is 185

本章主要内容



4.1 数学函数

4.2 字符数据类型和操作符

4.3 字符函数

4.4 字符串类型

4.5 格式化控制台输出

4.6 简单的文件输入输出

4.7 作业与实践



第04章作业



(1)正多边形的面积公式为
$$\text{Area} = \frac{n \times s^2}{4 \times \tan(\frac{\pi}{n})}$$

➤ 其中n为边数，s为边长。编写程序提示用户输入正多边形的边数和边长，输出其面积

(2)编写程序接收ASCII码（0~127之间的整数），输出其对应字符

(3)编写程序接收字符，输出其ASCII码

(4)假设字母a、e、i、o、u及其大写为元音。编写程序提示用户输入字母，判断字母是元音还是辅音

(5)编写程序提示用户输入1个大写字母，将其转换为小写字母

(6)编写程序提示用户输入五级制成绩字母a、b、c、d、f或其大写字母，输出字母对应的数值4、3、2、1、0

(7)编写程序提示用户输入0~15之间的整数，输出相应的十六进制数

■例如输入“11”时输出“B”



(8)编写程序随机生成一个带有3个大写字母的字符串

(9)编写程序提示用户输入3个城市的名称（拼音或英语），输出它们的升序排序结果

(10)编程提示用户输入ddd-dd-dddd形式的社会保障号码，其中d为一个数字，程序检查并显示输入是否有效

■学习后面章节中的循环语句后再优化算法